



דוח הגשה עבודה 3- ארדואינו

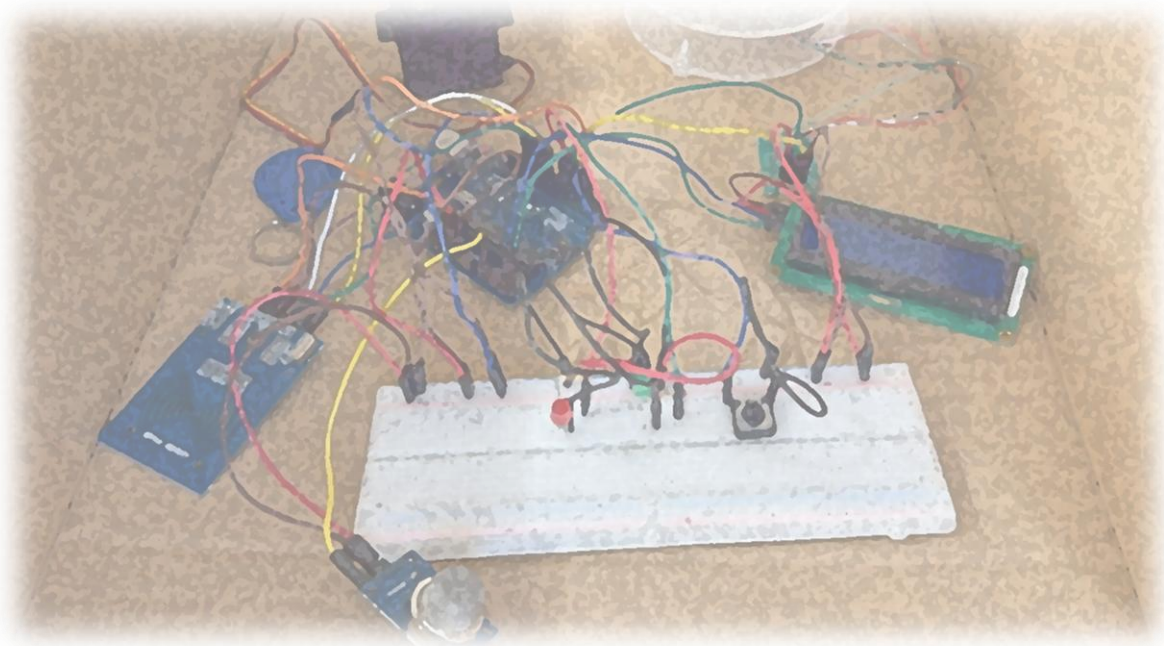
מספר קבוצה 6

322517251

315238626

323132282

208424382





1. תיאור מטרת המערכת ומשימת הבקרה

לאחר מאות שנים של עבודות ומכות קשות, בני ישראל אורזים את המצות בחיפזון ויוצאים אל המדבר בדרכם אל הארץ המובטחת. אולם, היציאה ההמונית מלווה בחששות כבדים מצד שלטונות מצריים, פרעה חושש מפני הברחה של אוצרות מצריים, כלי כסף, זהב ורכוש חורג, וכן מפני יצירת מהומות, התקהלויות מסוכנות או פריצות אלימות של מעברי הגבול.

כדי להתמודד עם אתגר מורכב זה, הקמנו, מהנדסי הממלכה באוניברסיטת בן-גוריון, תחנת מכס חכמה וממוחשבת מבוססת ארדואינו. המערכת משלבת זיהוי מבוסס RFID, שקילה מדויקת בזמן אמת ובקרת עוצמת הרעש במחסום, על מנת להבטיח שכל נוסע עובר תהליך בידוק קפדני, מסודר ומבוקר לפני פתיחת שער היציאה אל המדבר.

מטרת המערכת:

המערכת שיצרנו היא תחנת מכס חכמה וממוחשבת בשם "Pharaoh Customs", אשר נועדה לנהל ולבקר את יציאתם ההמונית של בני ישראל ממצרים בצורה מאובטחת, יעילה ומסודרת. הלוגיקה של פעולת המערכת מותאמת למסוף בידוק גבולות מתקדם ומיועדת לאמת תחילה את זהות הנוסעים באמצעות RFID, תוך מתן חיווי ויזואלי ומילולי מקיף על גבי מסך ה-LCD ומערך הנורות הן למפעיל התחנה והן לנוסע. בהמשך התהליך, המערכת מבצעת בקרה בטיחותית ותקציבית על משקל הכבודה של הנוסע באמצעות חיישן משקל מיוחד, על מנת לוודא שאף אחד לא מבריא איתו את אוצרות מצרים חלילה, ושהמשקל נשאר בטווח המותר. ברגע שהשער (מנוע הסרב) נפתח לרווחה למעבר, המערכת לא נחה ומפעילה מיקרופון המבצע מדידת רעש סביבתי באופן רציף, וכל זאת במטרה לזהות המולות, צעקות או ניסיונות פריצה בחסות הרעש. המערכת כולה תוכננה כך שבכל מקרה של חריגת משקל, תג שגוי או רעש חזק בשער היא תינעל אוטומטית, תדליק נורה אדומה ותגן על מעבר הגבול עד לאיפוס ידני של המפעיל.

משימת הבקרה והלוגיקה:

- שלב ההמתנה: המערכת מאפסת את השער למצב סגור, מכבה נורות חיווי ומציגה על מסך ה-LCD את ההודעה. "Pharaoh Customs - Scan Passport"
- שלב הזיהוי (RFID): המשתמש נדרש לסרוק תג. אם התג אינו מורשה או שגוי, המערכת נחסמת, נורת LED אדומה נדלקת ומוצגת הודעת דחייה.
- שלב השקילה: (HX711 + Load Cell): אם זהות האדם אושרה, המערכת ניגשת לקרוא נתונים מחיישן משקל דיגיטלי:
משקל תקין (עד 200 גרם): המטען מאושר, נורת LED ירוקה נדלקת ושער ה-Servo נפתח לרווחה.
משקל חורג קל (בין 200 ל-400 גרם): מוגדר כמטען כבד אך מאושר. המערכת מציגה אזהרה, אך מאפשרת את פתיחת השער בליווי נורה ירוקה.
חשד להברחה (מעל 400 גרם): המשקל מוגדר כלא תקין. המערכת מפעילה התרעת הברחה, נועלת את השער, מדליקה LED אדום ומשהה את הפעילות.
- שלב האבטחה הסביבתית (Microphone): בזמן שהשער פתוח ונוסעים עוברים, המערכת דוגמת את חיישן הקול. אם מזהה רעש חריג מעל לסף המוגדר-50 (חשד למהומה או הברחה), המערכת מבצעת נעילת חירום מיידית סוגרת את השער מיד ומקפיאה את המערכת ל-5 שניות עם חיווי אדום.
- שלב האיפוס: בסיום כל סבב בידוק, המערכת עוברת למצב המתנה ללחיצת כפתור פיזי על ידי המפעיל, לצורך מעבר מסודר לנוסע הבא.

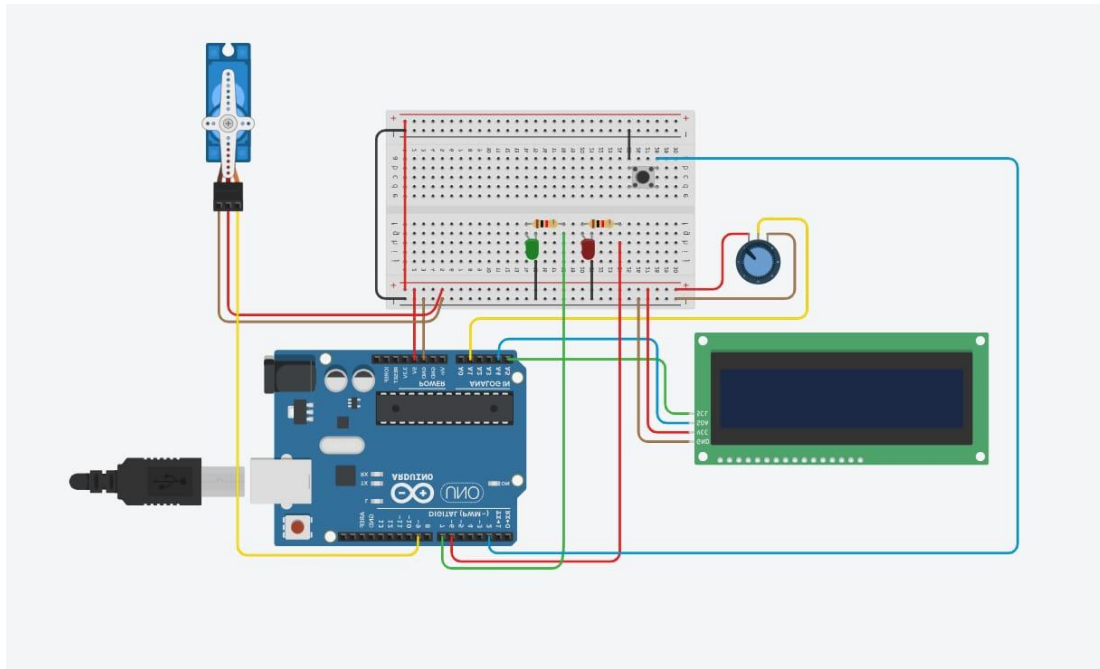
2. הפרויקט מומש באמצעות בקר ארדואינו אנו, והקוד נכתב בתוכנת IDE 2.3.9.

תיעוד הקוד מפורט בהמשך בנספח 1.

הרכיבים הכלולים בפרויקט הינם: נורת לד, חיישן מיקרופון, RFID, מסך LCD, כפתור לחיצה, מנוע HX711 Load Cell ו-SERVO.



התוכן נעשה בעזרת תוכנת הסימולטור Tinkercad כפי שניתן לראות בשרטוט הבא:



- השתמשנו ב- Potentiometer בתור חיישן המשקל.
- חסר מידול של RFID ומיקרופון מכיוון שלא היה קיים בתוכנה.



תיאור הרכיבים העיקריים:

מספר הפין	רכיב	הפעלה	תצורה
9,10,11,12	RFID	זיהוי תג הנוסע	Input / Output
4,5	HX711	מדידת משקל המטען	Input
A0	מיקרופון	ניטור רמת רעש	Input
3	Servo	שליטה על שער המעבר	Output
A4-A5	LCD	מסך תצוגת הודעות	Output
6	נורה אדומה	חיווי לאירוע חריג	Output
7	נורה ירוקה	חיווי למעבר תקין	Output
2	כפתור RESET	איפוס המערכת	Input

3. תעודת זהות לחיישן המיוחד (Load Cell & HX711) במערכת:

טווח המדידה: הטווח הדינמי המקסימלי של תא העומס הפיזי במעבדה מוגדר בין 0 ל-5 קילוגרם.

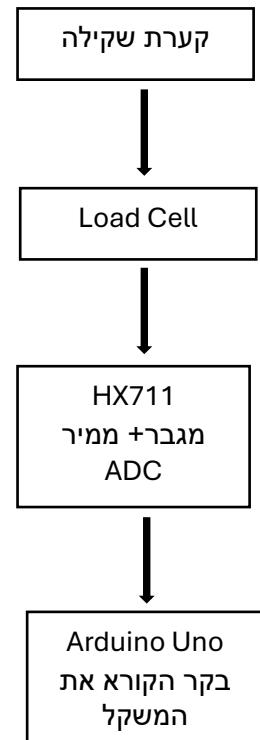
הטווח האופרטיבי במערכת: בקוד המערכת מוגדר טווח ניטור יישומי ממוקד הנע בין 0 ל-400 גרם ומעלה, המשמש לסיווג חריגות משקל בכניסה למסוף.

רזולוציית המרה: רכיב ה-HX711 כולל ממיר אנלוגי לדיגיטלי, בעל רזולוציה של 24 ביטים, המאפשר חלוקה תאורטית של אות המתח האנלוגי ל- 2^{24} רמות קלט דיגיטליות, ובכך מקנה למערכת רגישות גבוהה ודיוק רב.

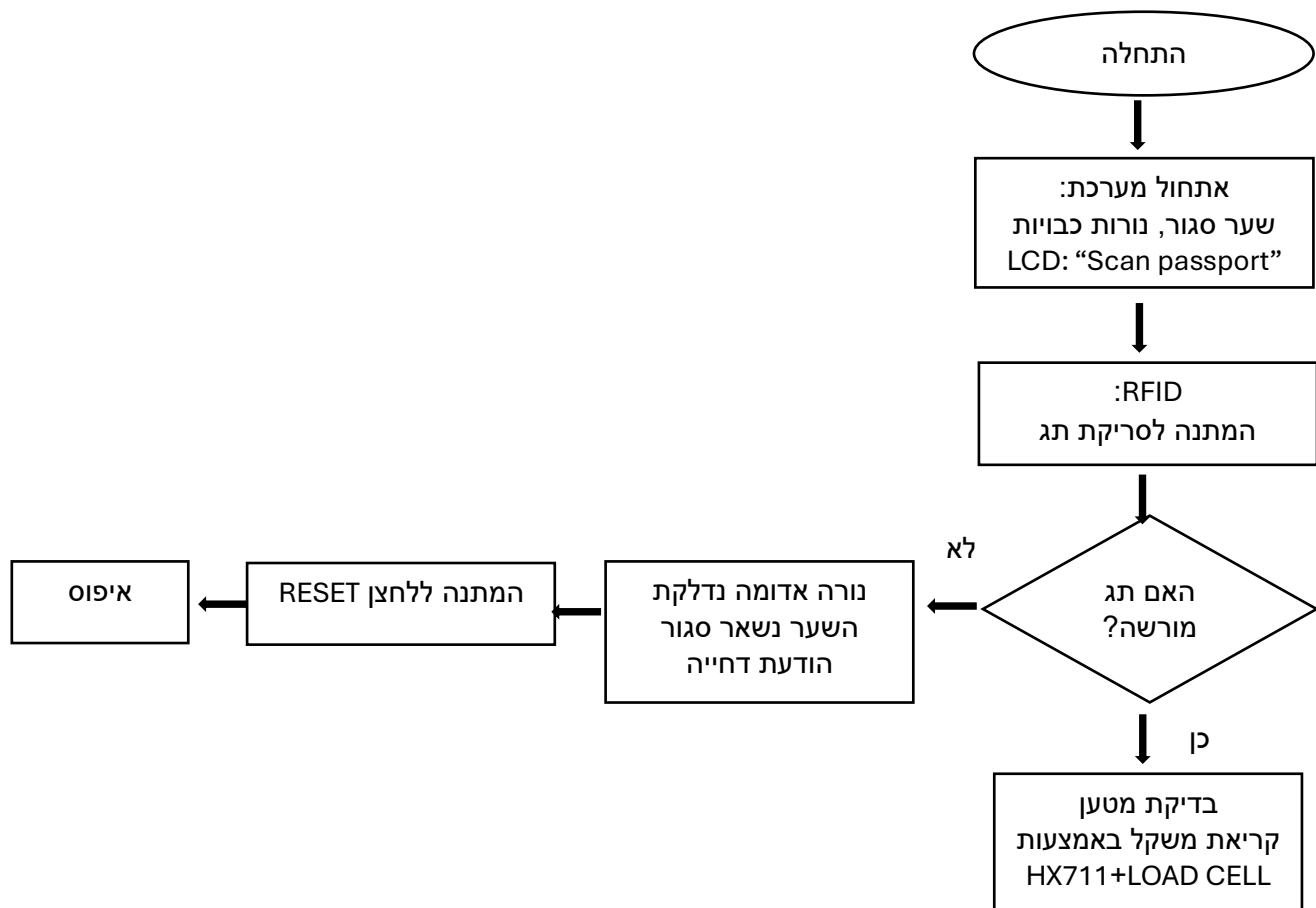
סוג אות הפלט: אות דיגיטלי טורי סינכרוני, המועבר אל מיקרו-בקר ה-Arduino באמצעות קו נתונים (DT) וקו שעון (SCK).

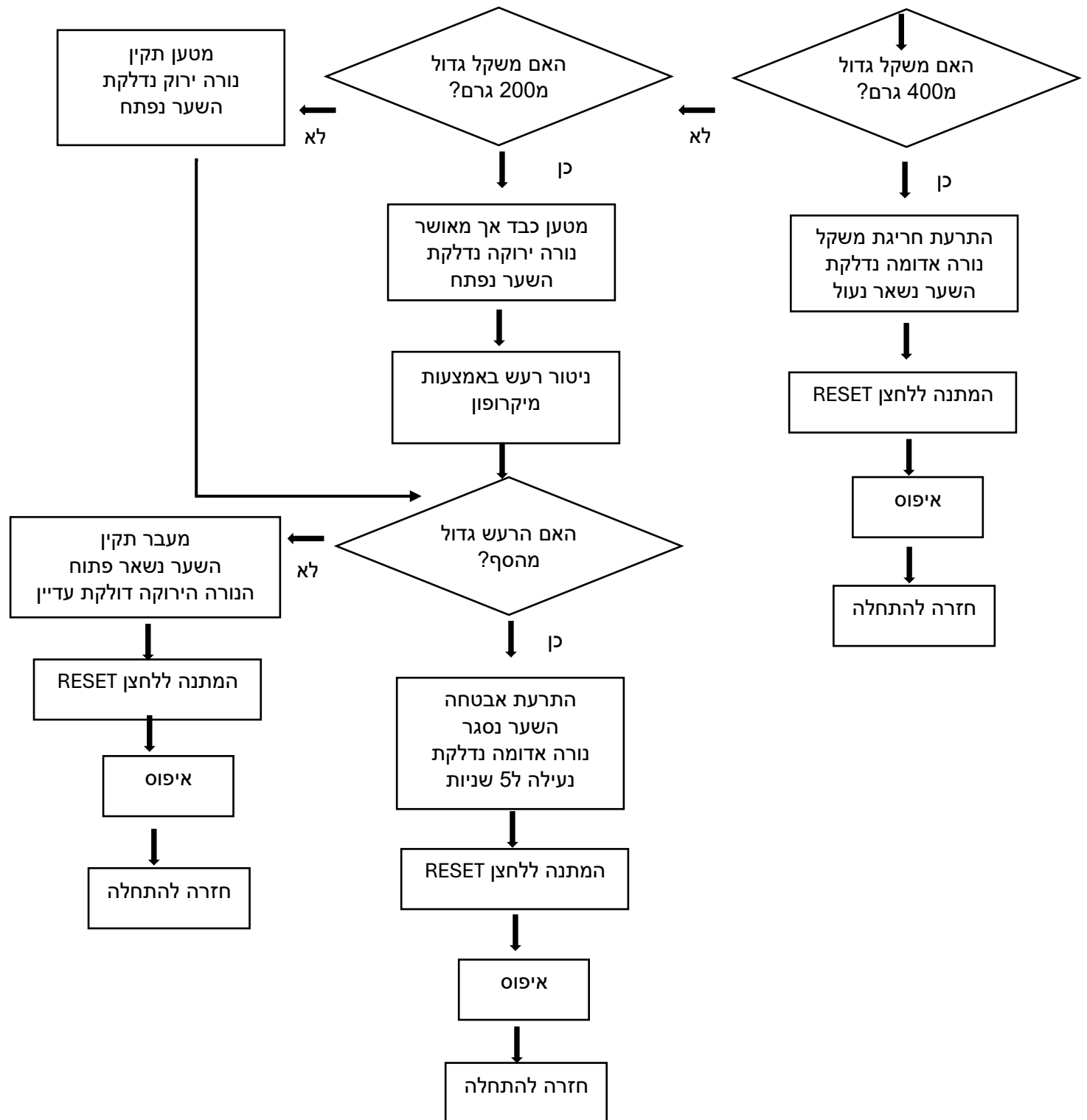
תהליך עיבוד האות: מאחר שהאות החשמלי הראשוני המופק מתא העומס הוא אות אנלוגי חלש ומזערי, שבב ה-HX711 מבצע שלב חיוני של הגברה ביחס של פי 64 או פי 128, לצד סינון רעשים, לפני ביצוע הדיגיטציה והעברת הנתונים לבקר.

פונקציית המרה וקליברציה: בהתאם לתהליך הכיול ההנדסי, כדי לתרגם את אות המוצא הדיגיטלי הגולמי ליחידות משקל פיזיקליות (גרמים), מיושמת בקוד פונקציית המרה מתמטית המבוססת על חלוקת הערך הנקרא במקדם כיול תוכנתי קבוע שהוגדר בניסוי:
calibration Factor = 392.76



4. תרשים זרימה של המערכת







5. בדיקת המערכת ב-5 תנאים שונים:

מספר בדיקה	תנאי הבדיקה	תיאור	תגובת המערכת	מסקנה
1	סביבה שקטה	המערכת הופעלה בחדר שקט יחסית, ללא דיבור חזק או תנועה רבה סביב המיקרופון	המערכת פתחה את השער ולא הופעלה התרעת רעש	בתנאי שקט המערכת יציבה
2	דיבור רגיל ליד המערכת	הופעלה שיחה רגילה במרחק בינוני מהמיקרופון בזמן שהשער פתוח	ברוב המקרים המערכת המשיכה לפעול כרגיל ללא נעילה	דיבור רגיל לא תמיד עובר את סף הרעש ולכן לא בהכרח גורם להתרעה
3	דיבור חזק קרוב למיקרופון	אדם דיבר בקול גבוה קרוב יחסית לחיישן המיקרופון	בחלק מההרצות המערכת זיהתה רעש חריג והפעילה התרעת אבטחה	המערכת רגישה יותר כאשר מקור הרעש קרוב למיקרופון
4	רעש פתאומי	בוצעה מחיאת כף או רעש חד בזמן שהשער פתוח	המערכת הגיבה במהירות, הדליקה נורה אדומה וסגרה את השער	הרגישות הגבוהה ביותר זוהתה ברעשים פתאומיים וחזקים
5	אזור עם תנועה ורעשי רקע	נבדקה בסביבה פעילה יותר- במסדרון באוניברסיטה.	התקבלו תגובות פחות יציבות, ולעיתים קריאת הרעש הייתה גבוהה יותר	רעשי רקע משתנים ותנועה סביב המערכת משפיעים על יציבות קריאת המיקרופון

מתי זוהתה רגישות במערכת?

מצאנו כי המערכת רגישה במיוחד לרעשים פתאומיים וחזקים, כמו מחיאת כף, נקישה על השולחן או דיבור חזק קרוב לחיישן. לעומת זאת, בסביבה שקטה או בזמן דיבור רגיל במרחק סביר, המערכת בדרך כלל לא הפעילה התרעת רעש. בנוסף, זיהינו כי כאשר המערכת פעלה בסביבה עם תנועה ורעשי רקע משתנים, קריאת המיקרופון הייתה פחות יציבה.

6. מסקנות מהעבודה ושדרוג אפשרי למערכת הבקרה

במהלך העבודה למדנו כיצד ניתן לשלב מספר רכיבי קלט ופלט במערכת בקרה אחת המבוססת על Arduino המערכת משלבת זיהוי משתמש באמצעות RFID, מדידת משקל באמצעות Load Cell וניטור רעש באמצעות מיקרופון, חיווי באמצעות נורות ומסך, LCD ושליטה פיזית על שער באמצעות מנוע SERVO. שילוב הרכיבים איפשר לנו ליצור מערכת המדמה תחנת מכס חכמה, שבה כל נוסע עובר תהליך בדיקה מסודר לפני פתיחת השער.

אחת המסקנות המרכזיות מהעבודה היא שהמערכת עובדת בצורה טובה כאשר תנאי הסביבה יציבים, אך קיימת רגישות גבוהה יחסית בחיישן המיקרופון. במהלך הבדיקות ראינו כי רעשים פתאומיים או רעשים



קרובים לחיישן עלולים לגרום להפעלת התרעת אבטחה. לכן, יש חשיבות רבה לבחירת סף רעש מתאים בקוד, כדי למנוע התרעות שווא מצד אחד, אך עדיין לזהות אירוע חריג מצד שני. בנוסף, למדנו כי חיישן המשקל דורש כיול מדויק ואיפוס לפני המדידה. כאשר המשקל מונח בצורה יציבה על משטח השקילה, הקריאה מתקבלת בצורה טובה יותר. לעומת זאת, הנחה לא יציבה או שינוי במיקום המטען עלולים להשפיע על הקריאה. לכן חשוב לבצע תהליך כיול מסודר ולהמתין רגע קצר עד שהקריאה מתייצבת. **שדרוג אפשרי למערכת הבקרה הוא הוספת כיול אוטומטי ודינמי לחיישן הרעש.** במקום להשתמש בסף רעש קבוע מראש, המערכת תוכל למדוד את רעש הרקע בתחילת כל הפעלה, לחשב רמת רעש בסיסית, ורק לאחר מכן לקבוע סף התרעה מתאים. כך, אם המערכת פועלת בסביבה שקטה הסף יהיה נמוך יותר, ואם היא פועלת בסביבה רועשת הסף יעלה בהתאם. שדרוג זה יכול להפחית התרעות שווא ולשפר את אמינות מערכת האבטחה.

7. נספחים

נספח 1- תיעוד הקוד

```
#include <SPI.h>
#include <MFRC522.h>

#include <HX711.h>

#include <Servo.h>

#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#define RFID_SS_PIN 10

#define RFID_RST_PIN 9

#define HX711_DT_PIN 4

#define HX711_SCK_PIN 5

#define SERVO_PIN 3

#define RED_LED_PIN 6

#define GREEN_LED_PIN 7

#define RESET_BUTTON_PIN 2

#define MIC_PIN A0
```




```
bool gatelsOpen = false;
```

```
const int SERVO_STOP = 90;
```

```
const int SERVO_OPEN_SPEED = 0;
```

```
const int SERVO_CLOSE_SPEED = 180;
```

```
const unsigned long GATE_MOVE_TIME = 1200;
```

```
const unsigned long GATE_WAIT_TIME = 2000;
```

```
const float MAX_NORMAL_WEIGHT = 200.0;
```

```
const float MAX_APPROVED_HEAVY_WEIGHT = 400.0;
```

```
const int NOISE_THRESHOLD = 50;
```

```
const unsigned long SECURITY_LOCK_TIME = 5000;
```

```
//Replace this UID with your real RFID tag UID from Serial Monitor.
```

```
const byte AUTHORIZED_UID[] = {0xAA, 0xA4, 0x20, 0xB0};
```

```
const byte AUTHORIZED_UID_LENGTH = sizeof(AUTHORIZED_UID);
```

```
//Must be calibrated for your load cell.
```

```
float calibrationFactor = 392.76;
```

```
MFRC522 rfid(RFID_SS_PIN, RFID_RST_PIN);
```

```
HX711 scale;
```

```
Servo gateServo;
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
void setup} ()
```

```
Serial.begin(9600)
```

```
pinMode(RED_LED_PIN, OUTPUT);
```



```
pinMode(GREEN_LED_PIN, OUTPUT);  
pinMode(RESET_BUTTON_PIN, INPUT_PULLUP);
```

```
gateServo.attach(SERVO_PIN);  
stopGate();
```

```
lcd.init();  
lcd.backlight();
```

```
SPI.begin();  
rfid.PCD_Init();
```

```
scale.begin(HX711_DT_PIN, HX711_SCK_PIN);  
scale.set_scale(calibrationFactor);  
scale.tare();
```

```
turnOffLeds();  
showMessage("Pharaoh Customs", "Scan Passport");
```

```
Serial.println("System started");  
Serial.println("Scan RFID tag");  
{  
void loop} ()  
  
if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial())  
    return;  
  
{  
  
Serial.print("Scanned UID: ");  
printUid(rfid.uid.uidByte, rfid.uid.size);
```



```
bool authorized = isAuthorizedTag(rfid.uid.uidByte, rfid.uid.size);

rfid.PICC_HaltA();
rfid.PCD_StopCrypto1();

if (!authorized){
    handleUnauthorizedTraveler();
    return;
}

showMessage("Welcome Traveler", "Checking Cargo");
delay;(1000)

float weight = readWeight();
Serial.print("Weight = ");
Serial.print(weight);
Serial.println(" grams");

decideAccess(weight);

waitForResetButton();
resetForNextTraveler();
{

int readNoiseLevel} ()
    int minValue = 1023;
    int maxValue = 0;

    unsigned long startTime = millis();

    while (millis() - startTime < 60)}
```



```
int value = analogRead(MIC_PIN);

if (value < minValue)}
    minValue = value;
{

if (value > maxValue)}
    maxValue = value;
{
{

return maxValue - minValue;
{

float readWeight} ()
if (!scale.is_ready())}
    Serial.println("HX711 not ready");
    return 0;
{

float weight = scale.get_units;(10)

if (weight < 0)}
    weight = 0;
{

return weight;
{

bool isAuthorizedTag(byte *uid, byte uidLength)}
if (uidLength != AUTHORIZED_UID_LENGTH)}
```



```
        return false;
    }

    for (byte i = 0; i < uidLength; i++){
        if (uid[i] != AUTHORIZED_UID[i]){
            return false;
        }
    }

    return true;
}

void decideAccess(float weight){
    turnOffLeds();

    if (weight > MAX_APPROVED_HEAVY_WEIGHT){
        showMessage("Smuggling Alert", "Gate Locked");
        digitalWrite(RED_LED_PIN, HIGH);
        Serial.println("Smuggling suspected");
        delay ;(3000)

    { else if (weight > MAX_NORMAL_WEIGHT) {
        showMessage("Heavy Cargo", "Approved");
        digitalWrite(GREEN_LED_PIN, HIGH);
        openGate();
        Serial.println("Heavy cargo approved");
        delay;(3000)

    { else}

        showMessage("Cargo Approved", "Safe Journey");
        digitalWrite(GREEN_LED_PIN, HIGH);
```



```
    openGate;()

    Serial.println("Cargo approved");

    delay;(3000)

{
{

void handleSecurityAlert(int noiseLevel){

    turnOffLeds;()

    showMessage("Security Alert", "Crowd Noise");

    closeGate;()


    digitalWrite(RED_LED_PIN, HIGH);

    Serial.print("High noise detected. Noise level = ");

    Serial.println(noiseLevel);

    Serial.println("System locked for 5 seconds");


    delay(SEcurity_LOCK_TIME);


    digitalWrite(RED_LED_PIN, LOW);

    digitalWrite(GREEN_LED_PIN, LOW);


    showMessage("Pharaoh Customs", "Scan Passport");


    Serial.println("Security lock ended");

    Serial.println("Scan RFID tag");

{

void handleUnauthorizedTraveler} ()

    showMessage("Access Denied", "Pharaoh Alert");

    digitalWrite(RED_LED_PIN, HIGH);
```



```
digitalWrite(GREEN_LED_PIN, LOW);
```

```
Serial.println("Unauthorized traveler");
```

```
waitForResetButton;()
```

```
resetForNextTraveler;()
```

```
{
```

```
void waitForResetButton} ()
```

```
showMessage("Press RESET", "Next Traveler");
```

```
Serial.println("Waiting for RESET button");
```

```
while (digitalRead(RESET_BUTTON_PIN) == HIGH){
```

```
int noiseLevel = readNoiseLevel;()
```

```
if (noiseLevel > NOISE_THRESHOLD){
```

```
handleSecurityAlert(noiseLevel);
```

```
showMessage("Press RESET", "Next Traveler");
```

```
Serial.println("Waiting for RESET button");
```

```
{
```

```
delay;(100)
```

```
{
```

```
delay;(250)
```

```
while (digitalRead(RESET_BUTTON_PIN) == LOW){
```

```
delay;(20)
```

```
{
```

```
{
```



```
void resetForNextTraveler} ()

  turnOffLeds;()

  closeGate;()

  showMessage("Ready For Next", "Traveler");

  delay;(2000)

  rfid.PCD_Init;()

  delay;(100)

  showMessage("Pharaoh Customs", "Scan Passport");

  Serial.println("System reset");

  Serial.println("Scan RFID tag");
{

void openGate} ()
  if (gateIsOpen) return ;
  gateServo.write(SERVO_OPEN_SPEED) ;
  delay(GATE_MOVE_TIME);
  stopGate;()
  gateIsOpen = true ;

// ← דגימת רעש רק כשהשער פתוח
  unsigned long startTime = millis;()
  while (millis() - startTime < GATE_WAIT_TIME){
    int noiseLevel = readNoiseLevel;()
    if (noiseLevel > NOISE_THRESHOLD){
      handleSecurityAlert(noiseLevel);
```




```
        return;

    {

        delay;(100)

    {

    {

void closeGate} ()

if (!gatelsOpen) return ;

gateServo.write(SERVO_CLOSE_SPEED) ;

delay(GATE_MOVE_TIME)      ;

stopGate;()

gatelsOpen = false          ;

{

void stopGate} ()

    gateServo.write(SERVO_STOP);

{

void turnOffLeds} ()

    digitalWrite(RED_LED_PIN, LOW);

    digitalWrite(GREEN_LED_PIN, LOW);

{

void showMessage(const char *line1, const char *line2)}

    lcd.clear;()

    lcd.setCursor;(0 ,0)

    lcd.print(line1);

    lcd.setCursor;(1 ,0)

    lcd.print(line2);

{
```



```
void printUid(byte *uid, byte uidLength){  
    for (byte i = 0; i < uidLength; i++){  
        if (uid[i] < 0x10){  
            Serial.print("0")  
        }  
  
        Serial.print(uid[i], HEX);  
  
        if (i < uidLength - 1){  
            Serial.print(" ")  
        }  
    }  
  
    Serial.println();  
}
```

הנחות:

1. מניחים את הכבודה על המשקל לפני שמעבירים את הדרכון!! המפעיל יודע איך להשתמש במערכת
2. לחכות עד שה LCD מציג לסרוק את PASSPORT לא לסרוק לפני!
3. המשקל מ 400 גרם לא תקין
4. טווח משקל בין 200-400 יש אזהרה אבל תקין
5. אחרי כל נוסע שעובר צריך ללחוץ על הכפתור על מנת להתחיל בדיקה לנסע הבא
6. השער כדיפולט סגור, ולכן כאשר יש חריגות ובעיות הוא לא יסגר מחדש כי הוא כבר סגור ולכן פשוט לא ייפתח, כנ"ל לגבי אם השער פשוט סגור - יש משתנה בוליאני שבדוק את המצב שלו ולפי זה פותח או סוגר
7. הדגימת רעש מתבצעת רק כאשר השער פתוח - מצב שבו נוסעים עוברים בשער (מטען מאושר, ממתין לריסט לקראת הנוסע הבא) ויש רעש גבוה, חשד שתתבצע הברחה בזמן הזה. כאשר השער סגור אין סיבה לחסום מחדש.

נספח 2- README

1. חבר את בקר ה־ Arduino למחשב בעזרת כבל USB והפעל את המערכת.
2. ודא שהשער סגור, נורות החיווי כבויות, ועל מסך ה־ LCD מופיעה הודעה לסריקת תג.
3. הנח מטען על משטח השקילה.
4. סרוק תג RFID של נוסע.
5. אם התג אינו מורשה, תידלק נורה אדומה והשער יישאר סגור. לחץ על כפתור RESET למעבר לנסע הבא.
6. אם המשקל תקין או כבד אך מאושר, תידלק נורה ירוקה והשער ייפתח.



7. בזמן שהשער פתוח, הימנע מרעש חזק ליד המיקרופון כדי לא להפעיל התרעת אבטחה.
8. במקרה של חריגת משקל או רעש חריג, תידלק נורה אדומה והשער ייסגר או יישאר נעול.
9. בסיום כל בדיקה, לחץ על כפתור RESET כדי לאפס את המערכת ולהכין אותה לנוסע הבא.

נספח 3- הצהרות שימוש בAI

בעבודה זו נעשה שימוש בChat Gpt, Gemini וClaude.